Seminar course

Verification in Systems

(aka “ver-sys”)

Preliminary meeting

https://dse.in.tum.de/

Prof. Reto Achermann

Course instructors

Systems Research Group

https://dse.in.tum.de/team/

Prof. Reto Achermann

Professor

ver-sys-seminar: Information

https://github.com/**TUM-DSE/seminars/**

Communication:

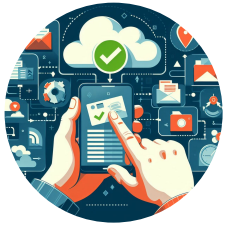
Join us with TUM email address (@tum.de)
ls1-courses-tum.slack.com
#ss26-ver-systems

Context

The Need for Correct Software Systems

Driving Assistants

Navigation



Cloud Storage

Payment Systems

Traffic and Grid Controls

Health Care

eCommerce

Currency Transfers

We rely on software systems every day

Building Safe & Secure Systems is Hard

Ideal World

We build systems that are safe,

Secure and performance.

"Systems that work as intended."

Reality

Regular reports about security issues.

Vulnerabilities occurring from

Misconfigurations, software bugs,

Hardware exploits

Human coding error identified as cause of digital bank service outage

The coding error in a software used for system maintenance had resulted in DBS Bank's hours-long service outage in May, but unrelated to another outage in March.

Researcher Finds Potholes In Vehicle Traffic Control Systems

Hundreds of thousands of road traffic sensors and repeater equipment are at risk of attack, researcher says.

Critical vulnerability affecting most Linux distros allows for bootkits

Buffer overflow in bootloader shim allows attackers to run code each time devices boot up.

Thunderclap flaws impact how Windows, Mac, Linux handle Thunderbolt peripherals

Thunderclap vulnerabilities allow the creation of highly dangerous malicious peripherals that can steal data from OS memory.

Buggy app for insulin-delivery device puts diabetes patients at risk of hypoglycemia

No fix available yet for over 100,000 Omnipod 5 customers

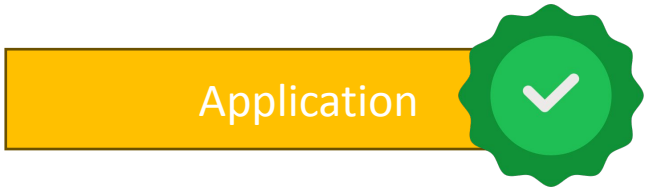
Tesla drivers report a surge in 'phantom braking'

Owners say their cars are suddenly slamming the brakes at high speeds, nearly causing crashes in many cases

Catastrophic software errors doomed Boeing's airplanes and nearly destroyed its NASA spaceship. Experts blame the leadership's 'lack of engineering culture.'

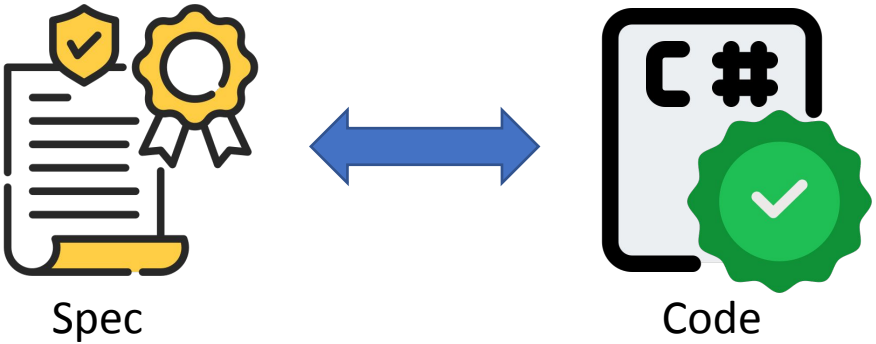
Question: how can we ensure, software systems behave as intended?

Correct Systems through Verification



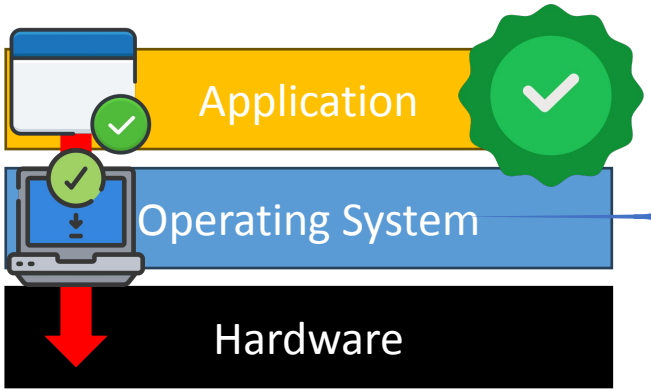
- **Verifying** the software application ensures correctness

- **Mathematical proof** that the application correctly implements the **specification**



Software Systems Rely on their Environment

- **Verifying** the software application ensures correctness

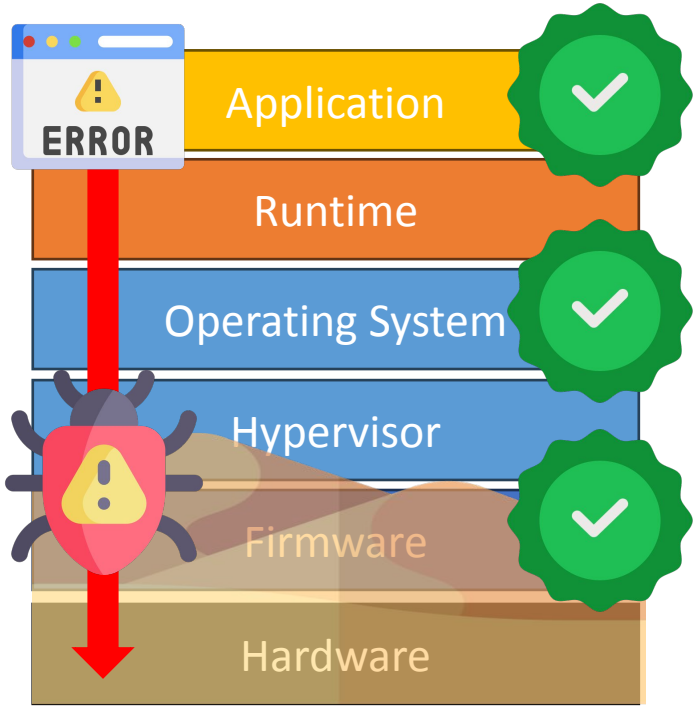


1) provides a **well-defined runtime environment** and isolation for the application

2) **initializes, configures and multiplexes hardware** (cores, memory, devices, …)

A correct application relies on the environment provided by the OS + Hardware

Software Systems Depend on Many Layers



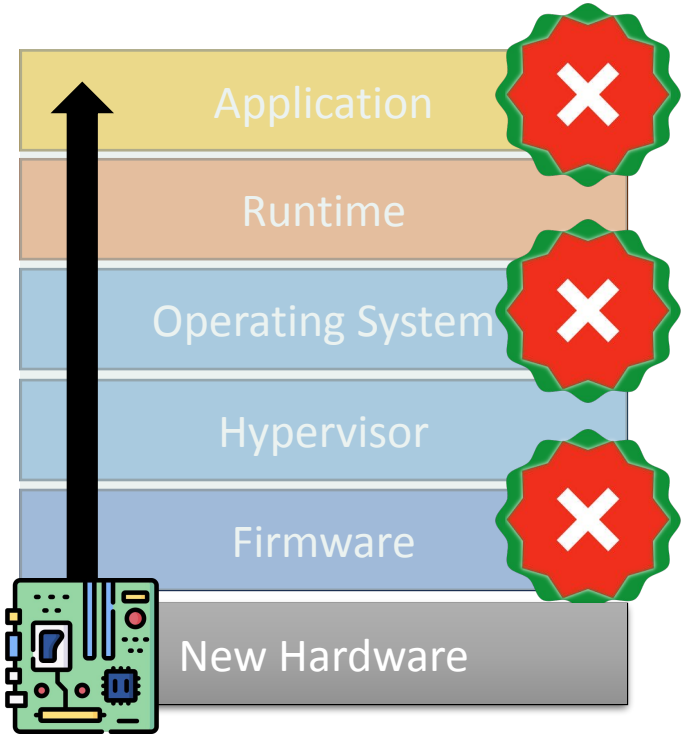
Verifying the application ensures correctness.

Unfortunately, this does not help much:

Each layer **depends on the runtime environment** provided by the layer below.

And there are many layers in today's Software Systems

Changes in the Environment trigger Re-Verification



Software verification:

- **significant effort**

  (e.g., 11+ person years for seL4 microkernel)

- **for a specific system configuration**

  (6 platforms: i.MX 6, i.MX8MM, Tegra TK1, Odroid XU4, HiFive, X86_64)

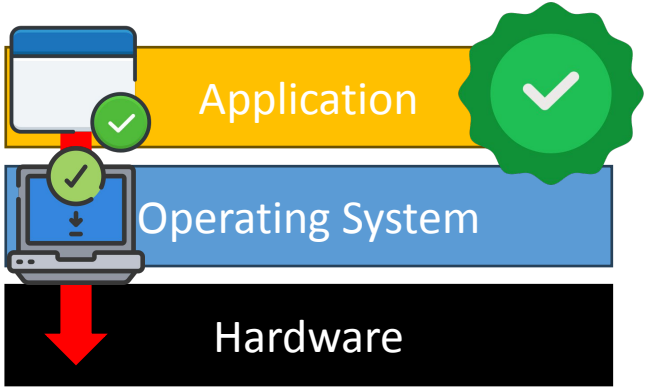
Adapt layers to the new execution environment (architecture, chip, …)

-> subtle differences & manual work

-> performance relevant

-> re-verification efforts

Challenges in System Verification



- **Scalability**: software systems can reach to million lines of code.
- **Concurrency**: multi-threading and other forms of parallelism
- **Memory Models**: when does a write become visible? Ordering of memory accesses.
- **Environment:** hardware features? E.g., MMU, SEV, TDX, …
- **3rd Party Libraries:** How can I integrate verified third-party code?

Format

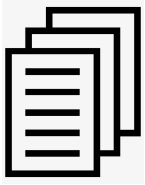
Tentative Topics

Verification focused papers from top systems conferences:
ASPLOS, OSDI, USENIX ATC, EuroSys, SOSP

Tentative topics
Kernel Verification
Handling concurrency
Push-Button Verification
Interactive theorem proving
…

Bird's eyes view

Team
(2 students per team)



Research papers
(Top systems conferences)

Understand

Research ideas

1 presentation

1 short report

Peer-reviewing

Overview

Phase I

Kick-off

Phase II: Understand & explore

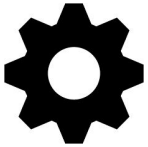
Understand

Presentation

Phase III: Research

Design

Implement (Bonus)



Phase IV: Report & review

Report

Peer-review

Phase I: Kick-off meeting

Format and motivation
(all participants meeting)

Team formation
(2 students per team)

Paper selection
(Top systems conferences)

The first week

NOTE

1. A list of papers will be provided for FCFS bidding
2. Paper presentation guidelines will be provided for the next phase

Phase II: Understand & explore

Understand the paper(s)

Focus

1. **Understand** the paper and related work
2. Also **explore** a "laundry list" of research ideas/directions

Paper presentation

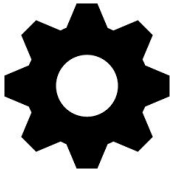
Focus

1. Explain the work/related work (**"why?"** and **"how?"**)
2. Explain and discuss all possible research directions
3. Pick a research direction

Phase III: Research

Research work

Focus:
Indepth research work to nail-down the problem and detailed approach to solve it!



Research prototype

Bonus:
(Optional)
"Build the system to solve it!" and show us the working idea and associated results

Phase IV: Report & review

Report

Focus

Prepare a single "short & sweet" report summarizing
(a) Paper
(b) Research work

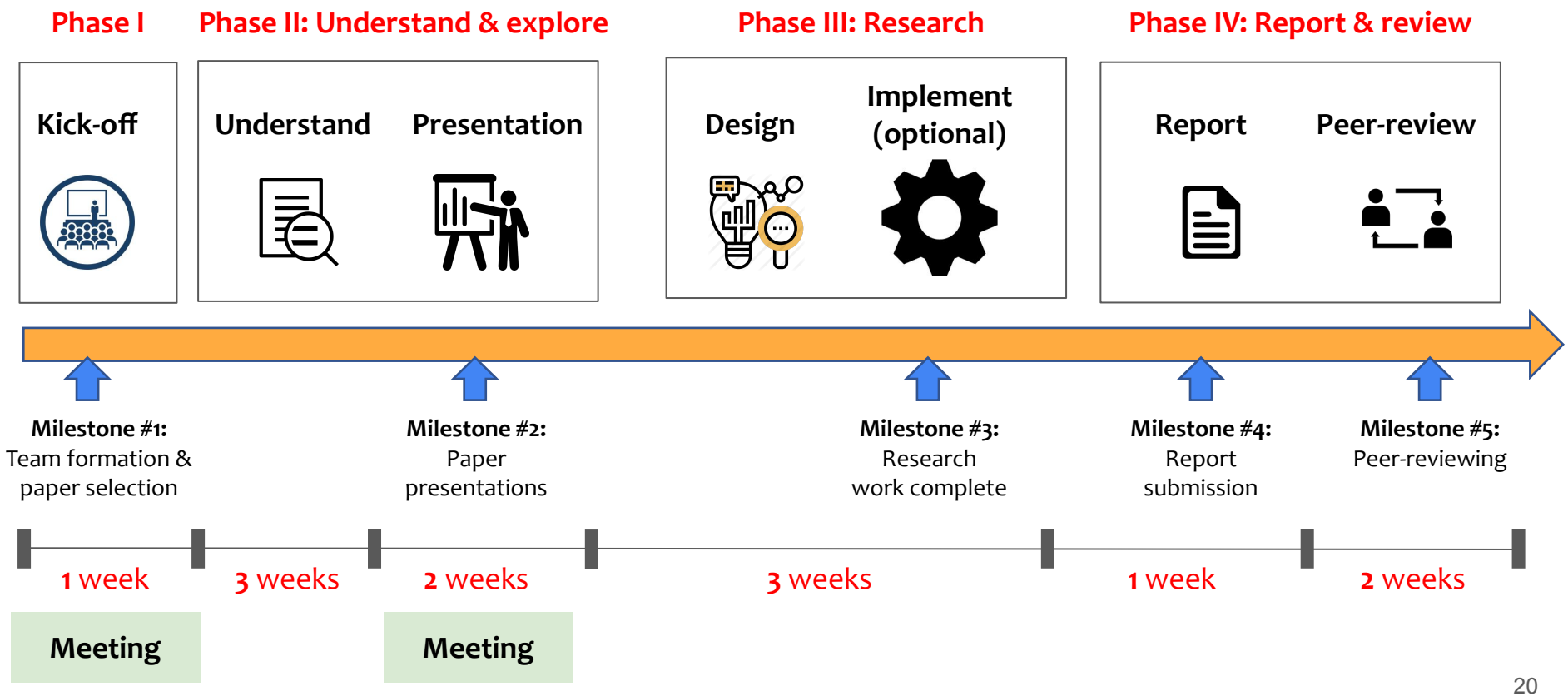
Peer-review

Focus

Give constructive (positive and critical) feedback for
(a) Paper summary
(b) Research work

END.

Overall timeline



Organization

- Format
  - Team-based seminar course (2 students per team)
- Communication
  - Slack for announcements and information sharing
  - Hotcrp for report submission and peer-reviewing
- Meetings (**in-person, attendance is compulsory**)
  - **Meeting #1:** Kick-off
  - **Meeting #2:** Paper presentation

Learning goals

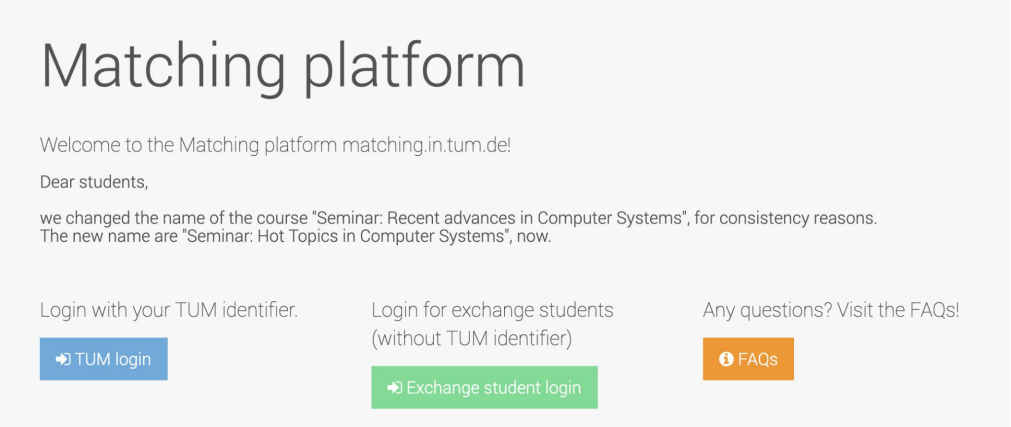
- Learn about the cutting-edge research in computer systems verification
- Promote critical thinking
- Cultivate an environment for innovation
  - To push the boundaries by advancing the state-of-the-art
- Improve scientific skills
  - Presentation
  - Writing
  - Communication: discussion and arguing
  - Mentorship: giving feedback and moderating discussion
- Encourage system building and evaluation
  - Learn by building, breaking, and benchmarking systems
- Importantly, to have fun!

Code of conduct

- University plagiarism policy
  - https://www.in.tum.de/en/current-students/administrative-matters/student-code-of-conduct/

- Decorum
  - Promote freedom of thoughts and open exchange of ideas
  - Cultivate dignity, understanding and mutual respect, and embrace diversity
  - Racism and bullying will not be tolerated

Interested?



Sign up on the TUM matching platform

Contacts

- Prof. Reto Achermann
  - reto.achermann@tum.de

- **All seminar-related info:** https://github.com/TUM-DSE/seminars

Workspace: http://ls1-courses-tum.slack.com/

Channel: #ss26-ver-systems

Join us with TUM email address (@tum.de)